Program 1 **Write a node.js script to enter one document to the collection after establishing a connection using mongoose. Raise exception and catch it, if any problem occurs. Print a proper message for error handling.**

Program 2 **Write a node.js script to enter more than one document to the collection after establishing a connection using mongoose. Raise exception and catch it, if any problem occurs. Print a proper message for error handling.**

Program 3(**Schema Validation)**

You are developing a MongoDB-based application using Mongoose. You need to define a userSchema that includes various validation rules to ensure data integrity and consistency.

1. **Define a Mongoose schema called userSchema with the following fields and validation requirements:**
    - **username:**
        - **Required and must be between 4 and 20 characters long.**
        - **Must start with letters and end with digits.**
        - **Should be trimmed of any leading or trailing spaces.**
        - **Should be converted to uppercase before saving.**
    - **email:**
        - **Required, must be unique across the collection.**
        - **Must follow the standard email format.**
    - **age:**
        - **Must be a number between 18 and 65.**
    - **role:**
        - **Must be either 'user' or 'admin'.**
        - **Should default to 'user' if not provided.**

Program 4 (**Validator**)

You are developing a MongoDB application using Mongoose and need to enforce specific validation rules on the email and product fields in your userSchema.

1. **Define a Mongoose schema userSchema with the following fields:**
    - **email:**
        - **The field is required and must be unique in the database.**
        - **It should be validated to ensure it contains a valid email address format.**
        - **If the provided email is invalid, the error message should indicate that the email address is not valid.**
    - **product:**
        - **The field is required.**
        - **It should only allow alphanumeric characters (i.e., letters and numbers).**
        - **If the product field contains invalid characters, the error message should indicate that it is not a valid alphanumeric code.**
2. **Write an asynchronous function createDoc that creates and saves a document with random data.**

Program 5 (**Validator**)

Write a node.js script to define a schema having fields like name, age, gender, email.

Apply following validations:

(1) Name field must remove leading/trailing spaces, minimum and maximum length should be 3 & 10 respectively, and name should be stored in lowercase

(2) Age must accept value greater than 0.

(3) Perform Email ID validation on Email field.

(4) Gender must accept values in uppercase only and allowed values are “MALE” & “FEMALE” only.

Program 6

Write a Node.js script using Mongoose to perform the following operations on the **movies** collection.

1. Insert multiple movie documents.
2. Display all movies having a rating greater than **8.5**.
3. Display the **title** and **rating** of the movie having the **second highest rating**.
4. Increase the rating of all **Action** movies by **0.2**.
5. Count the total number of **Hindi** movies.
6. Delete the movie having the title **"Jawan"**.

Program 7

Write a Node.js script using Mongoose to perform the following operations on the **courses** collection.

1. Insert multiple course documents.
2. Display the course having the highest fees.
3. Find the course named **"MERN Stack Development"**.
4. Update the fees to **₹15,000** and duration to **5 months** for the course named **"MERN Stack Development"** using **findByIdAndUpdate**.
5. Display all **Online** courses having fees less than **₹20,000**.
6. Display all **active** courses whose duration is greater than **4 months**, but exclude courses that are **Online** or belong to the **Cloud** category.
7. Delete the course named **"MERN Stack Development"** using its **_id**.

Program 8(Connectivity of MongoDB with ExpressJS/NodeJS using HTML)

Create html file which contains a form having username and password and insert data entered by user in collection named "data1".